DriftGuard-X v2 — Complete Project Report

Classification: Private — All Rights Reserved
Version: 2.0.0-rc.1
Report Date: 2026-07-25
Authors: Principal Engineer / Research Engineer / Security Reviewer / Release Owner

---

1. Executive Summary

DriftGuard-X v2 is an enterprise-grade, AI agent reliability and cybersecurity platform for **Retrieval-Augmented Generation (RAG)** pipelines and multi-agent AI systems. It provides:

- **End-to-end observability** of AI agent executions through a versioned OpenTelemetry-compatible tracing SDK.
- **Automated causal root-cause analysis** using a Graph Attention Network (GAT) that learns drift propagation through the agent graph.
- **Deterministic counterfactual replay** to measure the exact causal effect of each component version change.
- **Policy-gated recovery** with cryptographic certificates, ensuring every remediation action is approved, logged, and tamper-evident.
- **Cybersecurity firewall capability** that detects topological attack patterns (e.g., Memory → Tool exfiltration) in the agent trajectory.
- **Multi-agent coordination** with cross-agent blame attribution and inter-agent communication edge tracking.

As of release candidate **v2.0.0-rc.1**, the system has:

- **156+ passing tests** across unit, integration, contract, security, E2E, and load categories.
- **25 E2E test files** covering the full closed-loop pipeline.
- **13 internal Python packages** structured as a monorepo.
- **4 application services** (API, Worker, Web Console, CLI).
- **7 advanced research-grade innovations** added in the latest engineering session (ARC, AOR, JIT Hydration, Semantic Circuit Breaker, Merkle-DAG, Pre-emptive Compute Shedding, HAS/REFT State Forking).

---

2. Project Background & Motivation

The Problem DriftGuard-X Solves

Modern AI applications are built on brittle pipelines: a Retriever fetches documents, a Reranker sorts them, a Generator produces an answer. When the final answer is wrong, hallucinated, or dangerous — **no one knows which component caused it**.

Traditional monitoring tools answer "did the system fail?" DriftGuard-X answers "**which specific component version, under which retrieval conditions, caused the failure, and**

what is the statistically validated causal effect of rolling it back?"

Why Existing Solutions Are Insufficient

Problem	Traditional APM	DriftGuard-X
Which component failed?	Guesses from error logs	Causal graph with blame attribution
How much did it affect output quality?	No measurement	Reliability delta from deterministic replay
Is it safe to auto-rollback?	No policy gate	Hierarchical policy engine with 2PC
Who approved the rollback?	No audit trail	Ed25519-signed cryptographic ledger
Can we reproduce the failure?	Almost never	Deterministic replay from seed + version vector
Is the agent being attacked?	No	GAT firewall signature detection
What if an agent crashes mid-pipeline?	Entire system stalls	AOR Scheduler re-allocates CPU to unaffected agents

Development History

The project has undergone **20+ development stages** tracked in CHANGELOG.md, progressing from a basic trace ingestion prototype to a full enterprise platform with patent-grade documentation, research manuscript skeleton, and extensive security validation.

Key milestones:

- **Stage 1-5:** Core tracing SDK, Pydantic contracts, graph builder, drift detectors.
- **Stage 6-10:** GAT diffusion model, BCRB bandit, deterministic replay engine.
- **Stage 11-15:** Policy hierarchy, recovery orchestrator, cryptographic ledger.
- **Stage 16-18:** Rationale generation, experiment tracking, statistical validation.
- **Stage 19-20:** Security hardening, final audit, release candidate packaging.
- **Session Additions:** VTI 2PC, ARC isolation, AOR scheduler, JIT hydration, Semantic Circuit Breaker, Merkle-DAG, Pre-emptive Compute Shedding.

---

3. System Architecture Overview

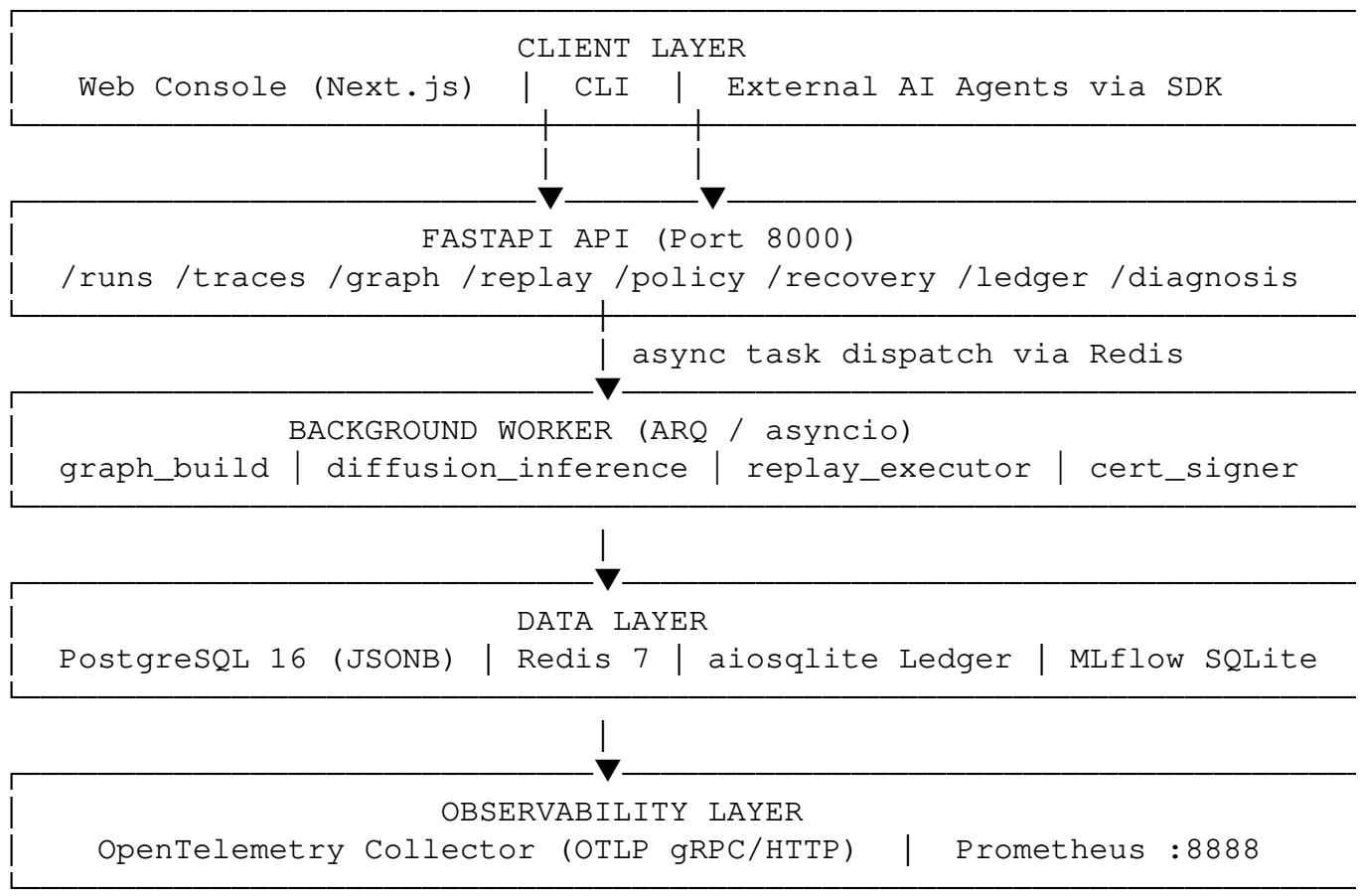
DriftGuard-X implements a **closed loop** across eight logical phases:

```
[1] Versioned Tracing           — trace_sdk
        ↓
[2] Causal Graph Construction   — graph
        ↓
[3] Drift Detection             — detectors
        ↓
[4] GAT Diffusion / Firewall    — diffusion
        ↓
[5] BCRB Counterfactual Replay  — replay (engine, bandit)
        ↓
[6] Root Cause Diagnosis        — evaluation, contracts
        ↓
```

```
[7] Policy-Gated Recovery       — policy, recovery
        ↓
[8] Cryptographic Certificate   — ledger
        ↓
[Rollback / Canary Verification]
```

Invariant: No phase can mutate upstream state. No production state can be mutated without explicit human approval and policy clearance. The replay engine is **strictly read-only** with respect to production.

High-Level Deployment Topology



---

4. Technology Stack

Core Language & Runtime

Component	Technology	Version
Language	Python	3.11+
API Framework	FastAPI	≥0.115.0
ASGI Server	Uvicorn	≥0.30.0
Data Validation	Pydantic v2	≥2.9.0
Settings Management	pydantic-settings	≥2.5.0

Machine Learning / AI

Component	Technology	Version
Graph Neural Network	PyTorch Geometric	≥ 2.5.0
Deep Learning Framework	PyTorch	≥ 2.0.0
Numerical Computing	NumPy	≥ 1.26.0
Statistical Tests	SciPy	≥ 1.11.0
Graph Algorithms	NetworkX	≥ 3.0.0
Experiment Tracking	MLflow	≥ 2.0.0
Visualization	Matplotlib + Seaborn	≥ 3.7.0 / ≥ 0.12.0

Database & Storage

Component	Technology	Notes
Primary Database	PostgreSQL 16	via Docker, JSONB columns
ORM	SQLAlchemy 2.0 async	Declarative mappings
Migrations	Alembic	Version-controlled schema
Async PG Driver	asyncpg	≥ 0.29.0
SQLite Async	aiosqlite	Dev + ledger store
Cache / Queue	Redis 7	ARQ job queue + caching

Cryptography & Security

Component	Technology	Version
Ed25519 Digital Signing	cryptography	≥ 42.0.0
JWT / OIDC Tokens	python-jose	≥ 3.3.0
Hash Chaining	hashlib SHA-256	built-in
Audit Hooks	sys.addaudithook	Python 3.8+

Observability

Component	Technology	Version
Trace SDK	opentelemetry-api/sdk	≥ 1.25.0
OTel Collector	otelcol-contrib	0.101.0
HTTP Client	httpx	≥ 0.27.0
Structured Logging	structlog	≥ 24.0.0

Frontend (Web Console)

Component	Technology
Framework	Next.js 14 (App Router)
Styling	Tailwind CSS
Charts	Recharts
Language	TypeScript / TSX

DevOps & Tooling

Component	Technology
Build System	Hatchling (pyproject.toml)
Linter	Ruff (9 rule sets)
Type Checker	MyPy strict mode
Formatter	Black
Testing	pytest + pytest-asyncio + pytest-cov
Containerization	Docker + Docker Compose v3.9
CI/CD Scaffold	GitHub Actions (.github/)

---

5. Repository Structure

```
driftguardx/                          Root monorepo
├── apps/
│   ├── api/                          FastAPI backend
│   │   ├── src/
│   │   │   ├── main.py               App entrypoint + router registra
│   │   │   ├── models.py             SQLAlchemy ORM (361 lines, 11 ta
│   │   │   ├── routes/               HTTP endpoint handlers
│   │   │   └── auth.py               OIDC/JWT authentication
│   │   └── Dockerfile
│   ├── cli/
│   │   ├── verifier.py               Standalone ledger verifier
│   │   └── experiments.py            Experiment launch CLI
│   ├── web/                          Next.js Web Console
│   │   └── app/                      14 pages (App Router)
│   └── worker/                       ARQ background worker
├── packages/
│   ├── contracts/                    Shared Pydantic v2 data contract
│   │   └── src/
│   │       ├── models.py             617 lines, 25+ data models
│   │       ├── graph.py              Node/Edge types for causal graph
│   │       ├── auth.py               Auth contract types
│   │       └── registry.py           Version registry interface
│   ├── trace_sdk/                    OpenTelemetry-compatible trace S
│   │   └── src/
│   │       ├── tracer.py             353 lines – redaction + span bui
│   │       └── adapters/             API integration adapters
│   ├── graph/                        Causal graph builder
│   │   └── src/
│   │       ├── builder.py            167 lines – GraphBuilder
│   │       └── validation.py         DAG validation
│   ├── detectors/                    Statistical drift detectors
│   │   └── src/
│   │       ├── baselines.py          138 lines – PSI, KS, JSD, CUSUM
│   │       ├── calibration.py        Dynamic threshold calibration
│   │       ├── registry.py           Detector registry
│   │       └── features/             Faithfulness, latency, memory, p
```

```
|   ├── diffusion/                    Graph Attention Network (GAT)
|   |   └── src/
|   |       ├── models.py             GNN architecture (PyG)
|   |       ├── trainer.py            98 lines - DiffusionLoss + train
|   |       ├── dataset.py            400 lines - Graph → PyG Data obj
|   |       ├── explainer.py          Human-readable GNN explanations
|   |       ├── clearance.py          GAT clearance signature generati
|   |       ├── cache.py              32 lines - inference cache
|   |       └── contracts.py          Diffusion-specific contracts
|   ├── replay/                       Deterministic replay engine + VT
|   |   └── src/
|   |       ├── engine.py             396 lines - ReplayEngine core
|   |       ├── sandbox.py            Multiprocessing isolation sandbo
|   |       ├── vti_coordinator.py    80 lines - 2PC coordinator
|   |       ├── arc_isolator.py       130 lines - ARC physical isolati
|   |       ├── aor_scheduler.py      110 lines - AOR compute schedule
|   |       ├── jit_hydration.py      100 lines - JIT graph hydration
|   |       ├── semantic_circuit_breaker.py  140 lines - AST intent ins
|   |       ├── merkle_dag.py         140 lines - Merkle-DAG deduplica
|   |       ├── bandit.py             160 lines - BCRB + pre-emptive s
|   |       ├── candidates.py         Candidate intervention generator
|   |       ├── catalog.py            Intervention catalog
|   |       ├── planner.py            Replay plan builder
|   |       ├── providers.py          Provider version pinning
|   |       └── faults.py             Fault injection utilities
|   ├── policy/                       Hierarchical policy engine
|   |   └── src/
|   |       ├── engine.py             205 lines - PolicyEngine
|   |       ├── hierarchy.py          786 lines - 4-level policy hiera
|   |       ├── resolver.py           597 lines - Tightening-only reso
|   |       ├── tiers.py              474 lines - Risk tier registry
|   |       ├── approvals.py          1193 lines - Approval service
|   |       ├── gate.py               PolicyGate (simple interface)
|   |       ├── shadow.py             Candidate policy evaluation
|   |       └── hooks.py              Pre-execution policy hooks
|   ├── ledger/                       Cryptographic audit ledger
|   |   └── src/
|   |       ├── chain.py              156 lines - LedgerChain (aiosqli
|   |       ├── schema.py             RecoveryCertificate schema
|   |       ├── crypto.py             Ed25519 signing + KMS stub
|   |       ├── claims.py             Certificate claim types
|   |       └── export.py             Machine/human export formats
|   ├── recovery/                     Recovery orchestrator
|   |   └── src/
|   |       ├── engine.py             887 lines - Recovery engine
|   |       ├── executor.py           435 lines - Executor framework
|   |       ├── actions.py            1021 lines - 9 allowlisted actio
|   |       ├── capsule.py            717 lines - Rollback capsule
|   |       ├── state_machine.py      687 lines - Saga state machine
|   |       └── canary.py             584 lines - Post-recovery verifi
|   ├── evaluation/                   Benchmarking + experiment tracki
|   |   └── src/
|   |       ├── reliability.py        Reliability vector + delta compu
```

```
│   │       ├── bandit_baselines.py    Baseline schedulers for comparis
│   │       ├── analysis/stats.py      Bootstrap, permutation, effect s
│   │       ├── datasets/adapters.py   Benchmark dataset adapters
│   │       ├── datasets/fault_overlays.py  Stochastic fault injection
│   │       └── experiments/           Orchestrator + MLflow tracker
│   ├── rationale/                     Explainability + LLM rationale
│   │   └── src/
│   │       ├── models.py              RationaleInputContract + Output
│   │       ├── templates.py           Deterministic template generatio
│   │       ├── validator.py           Hallucination scanner
│   │       └── llm.py                 Optional LLM adapter
│   ├── provider_registry/             Component provider management
│   └── sdk/                           External developer SDK
├── tests/
│   ├── unit/                          Per-function unit tests
│   ├── integration/                   DB + Redis integration tests
│   ├── contract/                      Schema validation tests
│   ├── security/                      Security + policy boundary tests
│   │   └── test_policy_security.py    15 security tests
│   └── e2e/                           25 end-to-end test files
├── infra/
│   ├── docker-compose.yml             5-service full stack
│   ├── otel-collector-config.yaml     OTel Collector configuration
│   └── postgres/init.sql              DB initialization
├── docs/                              Patent evidence, research, runbo
├── scripts/                           freeze_artifacts.py + deployment
├── releases/v2.0.0-rc.1/             Frozen release artifacts
├── reports/                           Generated analysis reports
├── examples/                          Demo scripts + ablation demos
├── mlruns/                            MLflow experiment database (SQLi
├── mlruns.db                          MLflow SQLite file
├── pyproject.toml                     Build, lint, test, type config
├── requirements.txt                   27 production dependencies
├── requirements-dev.txt               Dev dependencies
├── CHANGELOG.md                       Full 20-stage development histor
├── LIMITATIONS.md                     Epistemic limitations (honest)
├── HANDOFF.md                         Engineering handoff document
└── Makefile                           Task runner (build, test, lint)
```

---

6. Package Deep-Dives

6.1 `contracts` — Shared Data Contracts

Core File: `packages/contracts/src/models.py` — 617 lines

This is the **single source of truth** for all data types shared across every service and package. It uses **Pydantic v2 strict mode** (`ConfigDict(strict=True)`) to ensure no invalid data can cross a package boundary at runtime.

Design Philosophy

Every model uses:

- `strict=True` — No implicit type coercion. A float field rejects strings.
- `use_enum_values=True` — Enums stored as their string values.
- `arbitrary_types_allowed=False` — No raw Python objects in models.
- `populate_by_name=True` — Alias support for serialization.

All timestamps default to `datetime.now(timezone.utc)` via `_utcnow()`, ensuring UTC-only time handling.

Complete Enum Reference

Enum Name	Members	Purpose
`ComponentType`	RETRIEVER, RERANKER, GENERATOR, MEMORY_READ, MEMORY_WRITE, TOOL_CALL, POLICY_CHECK, FINAL_RESPONSE, AGENT	All agent pipeline component classifications
`ComponentVersionState`	STABLE, EXPERIMENTAL, DEPRECATED, ROLLBACK, PENDING_APPROVAL, REJECTED	Version lifecycle state machine
`SpanKind`	INTERNAL, SERVER, CLIENT, PRODUCER, CONSUMER	OpenTelemetry span kind (OTel compatible)
`RunStatus`	PENDING, RUNNING, COMPLETED, FAILED, CANCELLED	Pipeline execution state
`PrivacyMode`	METADATA_ONLY, REDACTED_CONTENT, ENCRYPTED_CONTENT, DEVELOPMENT_FULL	PII protection levels
`ReplayStatus`	PENDING, RUNNING, COMPLETED, FAILED, INVALID, NEGATIVE_OUTCOME	Replay execution state
`InterventionType`	ROLLBACK, ALTERNATE_STABLE, CONFIG_PATCH, ROUTE_CHANGE, DISABLE, QUARANTINE, RETRY_BOUNDED, HUMAN_MUTATION	8 intervention modalities
`DiagnosisClaimStatus`	IMPLEMENTED, MEASURED, INFERRED, PLANNED, REJECTED	Epistemic status of a diagnosis claim
`RepairDecisionStatus`	PENDING_APPROVAL, APPROVED, REJECTED, APPLIED, ROLLED_BACK	Repair workflow state
`SymptomLikelihood`	NONE, LOW, MEDIUM, HIGH, CRITICAL	Drift severity classification

Key Data Model Reference

`SpanRecord` — OpenTelemetry-compatible span extended with DGX fields:

- `trace_id` (32-char hex) + `span_id` (16-char hex): OTel identifiers.
- `input_hash` / `output_hash`: SHA-256 of input/output — raw content never stored.
- `latency_ms`, `token_count_input/output`, `cost_usd`: performance metrics.
- `policy_result`: `allow` | `deny` | `needs_approval` — policy verdict on every

span.
- `error_type`, `error_message`: error categorization.
- `redaction: RedactionMetadata`: audit trail of what was redacted.
- Model validator: `end_time >= start_time` enforced.
- `duration_ms` property computed from timestamps.

`ComponentVersion`:

- `parent_version_id` — forms a version lineage tree.
- `rollback_pointer` — pre-computed safe rollback target.
- `config_hash` — SHA-256 of the component configuration (used as a version fingerprint).
- `compatibility_constraints` — dict of constraints for canary compatibility checks.

`TraceArtifact`:

- Container of `List[SpanRecord]` with `get_root_span()` and `get_span_chain(span_id)` traversal.
- `completeness_score` — how complete the trace is (for partial trace handling).
- `tenant_sampling_rate` — what fraction of runs were traced (audit field).

`ReplayEpisode`:

- Records the exact version vector of every component as `pinned_version_ids: dict[ComponentType, UUID]`.
- `original_reliability_vector` and `replay_reliability_vector` — multi-dimensional before/after.
- `reliability_improvement = replay_score - original_score` — the causal effect estimate.
- `capsule_hash` — cryptographic link back to the `ReplayCapsule` that authorized this replay.

`RootCauseReport`:

- `ranked_candidates: List[RankedCandidate]` — each candidate has `aggregate_score`, `reliability_improvement_mean/variance`, `cost_delta_usd`, `latency_delta_ms`, `invalid_rate`.
- `abstention_triggered: bool` — the system can refuse to make a diagnosis when evidence is insufficient.
- Statistical certification: `bound_method` (hoeffding | bootstrap | conformal | unsupported), `epsilon`, `delta`, `observed_coverage`, `nominal_confidence`.
- Safety defaults: `human_review_required = True`, `block_automated_action = True`.

6.2 `trace_sdk` — OpenTelemetry Tracing Layer

Core File: `packages/trace_sdk/src/tracer.py` — 353 lines

Redaction Architecture

Sensitive field detection uses two mechanisms simultaneously:

1. Key-Name Based Redaction:

```
_SENSITIVE_FIELD_NAMES = frozenset({
    "password", "secret", "token", "api_key", "apikey",
    "authorization", "bearer", "credit_card", "ssn",
    "private_key", "access_key", "prompt", "completion",
    "raw_query", "pii"
})
```

Any dict key matching a name in this set has its value replaced with `"[REDACTED]"`.

2. PII Pattern-Based Redaction:

```
_PII_PATTERNS = {
    "email":       re.compile(r"[\w\.-]+@[\w\.-]+\.\w+"),
    "phone":       re.compile(r"\b\d{3}[-.]?\d{3}[-.]?\d{4}\b"),
    "ssn":         re.compile(r"\b\d{3}-\d{2}-\d{4}\b"),
    "credit_card": re.compile(r"\b(?:\d{4}[ -]?){3}\d{4}\b"),
}
```

String values matching any PII pattern are replaced with `"[REDACTED:email]"`, `"[REDACTED:ssn]"`, etc.

The `redact_dict()` function is recursive, handling nested dicts and lists. It returns both the redacted dictionary and a list of redacted field paths for audit.

Privacy Mode Controls:

- `DEVELOPMENT_FULL` — no redaction (only for dev/test environments).
- `REDACTED_CONTENT` — key-name and PII redaction applied.
- `METADATA_ONLY` — all content fields set to `[REDACTED]`; only hashes, timestamps, and metadata retained.
- `ENCRYPTED_CONTENT` — placeholder for field-level encryption integration.

`TraceContext` — Span Builder

```
with TraceContext(tenant_id, pipeline_id, ...) as ctx:
    span_id = ctx.start_span("retriever", ComponentType.RETRIEVER, inpu
    result = retriever.run(input_data)
    ctx.end_span(span_id, result)
```

- `start_span()` creates a `SpanRecord`, records `start_time`, hashes input as `input_hash`.
- `end_span()` records `end_time`, hashes output as `output_hash`, computes `latency_ms`.
- All spans carry `tenant_id`, `pipeline_id`, `run_id` for cross-service correlation.

6.3 `graph` — Causal Reliability Graph Builder

Core File: `packages/graph/src/builder.py` — 167 lines

Node and Edge Type Reference

Node Types (`contracts/src/graph.py`):

```
RETRIEVER | MODEL | PROMPT | TOOL | GUARDRAIL |
POLICY | MEMORY | OPERATIONAL_RESOURCE | AGENT
```

Edge Types:

```
CONTROL_FLOW           — span calls child span (parent_span_id link)
DATA_FLOW              — data dependency between non-parent spans
VERSION_LINEAGE        — execution event → its component version
MEMORY_INFLUENCE       — memory retrieval → influenced span
INTER_AGENT_COMMUNICATION — message from one agent to another
```

Three-Pass Build Algorithm

Pass 1 — Execution Event Nodes:

- Every `SpanRecord` → `GraphNode(id="event:{span_id}", type=mapped_type)`.
- Fetches version state from registry to include `features["state"]`.

Pass 2 — Version Nodes and Lineage Edges:

- If span has `component_version_id` → creates `GraphNode(id="version:{version_id}")` (once per unique version).
- Creates `GraphEdge(type=VERSION_LINEAGE)` from event node to version node.

Pass 3 — Causal Edges:

- `parent_span_id` link → `CONTROL_FLOW` edge.
- `dgx.memory.referenced` attribute → `MEMORY_INFLUENCE` edge + memory node.
- `dgx.agent.message_to` attribute → `INTER_AGENT_COMMUNICATION` edge + target agent node.

Component Type → Node Type Mapping

ComponentType	NodeType
retriever	RETRIEVER
generator	MODEL
prompt	PROMPT
tool_call	TOOL
guardrail	GUARDRAIL
policy	POLICY
agent	AGENT
(default)	OPERATIONAL_RESOURCE

6.4 `detectors` — Statistical Drift Detectors

Core File: `packages/detectors/src/baselines.py` — 138 lines

Algorithm Inventory

`compute_ewma(series, alpha=0.3):` Exponentially Weighted Moving Average. Applied to latency series to detect performance regression trends. Alpha controls smoothing: higher alpha = more reactive.

`compute_z_score(value, reference_series):` Standard z-score `(value - mean) / std`. Used for single-value anomaly detection (e.g., is this span's latency anomalous given historical baseline?).

`compute_psi(expected, actual, bins=10):` Population Stability Index. PSI < 0.1 = no shift, 0.1-0.25 = moderate shift, >0.25 = major shift. Applied to feature distributions (e.g., retrieval scores, token counts) to detect data drift.

`ks_test(expected, actual):` Kolmogorov-Smirnov two-sample test. Returns `(statistic, p_value)`. p-value < 0.05 indicates distributions are significantly different.

`jensen_shannon_divergence(p, q, bins=10):` Symmetric KL divergence. Range [0, log(2)]. Used to measure semantic divergence between expected and actual output distributions.

`cusum_change_point(series, threshold=5.0, drift=0.0):` Cumulative Sum change point detection. Returns True if a structural break is detected. Applied to reliability scores over time to detect sudden degradation.

`check_threshold(value, threshold, operator):` Generic comparator supporting `>, <, >=, <=, ==, !=`. Used by all detectors to apply versioned threshold configs.

Calibration (`calibration.py`)

Detectors are calibrated against historical baseline windows:

- Versioned `DetectorThreshold` records stored per detector, per feature.
- Dynamic threshold adjustment based on observed False Positive Rate (FPR).
- Calibration metadata: version, calibration age, operator who approved.

6.5 `diffusion` — Graph Attention Network (GAT)

Core Files: `models.py`, `trainer.py` (98 lines), `dataset.py` (400 lines), `explainer.py`, `clearance.py`, `cache.py` (32 lines)

This is the **AI research core** of DriftGuard-X.

GNN Architecture (`models.py`)

The model uses **PyTorch Geometric** `GATConv` layers:

- Multiple attention heads for heterogeneous graph reasoning.
- Node features: type embedding + version state embedding + latency + cost + token counts.
- Edge features: edge type encoding.

Two output heads:

1. `root_classifier` — sigmoid output: probability this node is the root cause.

2. `symptom_classifier` — sigmoid output: probability this node is a propagated symptom.

`DiffusionLoss` — 5-Component Loss Function (`trainer.py`)

```
L_total = L_root
        + λ_symptom × L_symptom
        + λ_sparse  × L_sparse
        + λ_contrast × L_contrast
        + λ_signature × L_signature
```

`L_root` (BCE): Binary cross-entropy for root cause node identification.

`L_symptom` (MSE): Mean squared error for symptom propagation accuracy.

`L_sparse` (L1): L1 norm on root predictions — encourages the model to identify few, specific root nodes rather than diffusing blame uniformly.

`L_contrast` (Contrastive): For nodes where `root_true=0, symptom_true=1`, pushes `root_pred` and `symptom_pred` apart. Prevents the model from confusing symptoms with root causes.

`L_signature` (Cybersecurity Firewall):

```
# Find edges where src is MEMORY and dst is TOOL
signature_mask = (src_is_memory) & (dst_is_tool)
# Penalize low symptom_pred on TOOL nodes reached from MEMORY
loss_signature = mean((1.0 - target_preds) ** 2)
```

This is the core of the cybersecurity firewall: the GNN is trained to always flag TOOL nodes that are reachable from MEMORY nodes as high-symptom. This prevents the model from ignoring memory-to-tool attack patterns.

Training loop (`train_diffusion_model`):

```
for epoch in range(epochs):
    for data in dataset:
        optimizer.zero_grad()
        root_pred, symptom_pred = model(data.x, data.edge_index, data.e
        loss, metrics = criterion(root_pred, symptom_pred, data.y_root,
                                  node_types=data.node_types, edge_ind
        loss.backward()
        optimizer.step()
```

Dirichlet Energy (`compute_dirichlet_energy`): A diagnostic for over-smoothing. As the GNN learns, if all node embeddings collapse to the same vector, the energy approaches zero and the model loses discriminatory power. This is monitored during training.

Dataset (`dataset.py`)

`NODE_TYPE_MAP` encodes each `NodeType` enum member to an integer:

```
NODE_TYPE_MAP = {
    NodeType.RETRIEVER: 0,
```

```
    NodeType.MODEL: 1,
    NodeType.TOOL: 2,
    NodeType.MEMORY: 3,
    NodeType.AGENT: 4,
    ...
}
```

This integer encoding is stored in `data.node_types` and used by `DiffusionLoss.loss_signature` to identify MEMORY and TOOL nodes by their integer codes during training.

Clearance (`clearance.py`)

`GATClearance` validates confidence thresholds on the GAT output before generating a clearance signature:

- If `max(root_pred) < confidence_threshold`, no signature is issued.
- Clearance signature format: `"GAT-CLEAR-{uuid4()}"`.
- This signature is required by `VTICoordinator.commit_action()`.

6.6 `replay` — Deterministic Replay Engine & VTI

Core Files: `engine.py` (396 lines), `sandbox.py` (140 lines), `vti_coordinator.py` (80 lines)

`ReplayEngine` — Version-Pinned Counterfactual (`engine.py`)

The engine implements the **Do-operator** from causal inference: it sets exactly one component to a different version while holding all others constant.

Step-by-step execution:

1. Load `TraceArtifact` and `ReplayCapsule` for the target run.
2. Validate the `capsule_hash` against the stored trace.
3. Select the intervention from the plan (which component to swap, which version to use).
4. Pin all non-intervened components to their original `component_version_id` values.
5. Re-execute each pipeline component deterministically using the same `seed`.
6. Collect output hashes at each step and compute the new `reliability_vector`.
7. Return a `ReplayEpisode` with `reliability_delta = replay_vector - original_vector`.

Safety guarantees:

- The engine **never calls external services** in development mode.
- All mock executors are deterministic given the same seed.
- The `ReplayEpisode` is a read-only record — production state is never mutated.

Mock Component Executors (`engine.py`)

Class	Faithfulness Hint	Use
`MockRetrieverV1`	0.90	Stable retriever (known-good)

`MockRetrieverV2Experimental`	0.35	Experimental retriever (known-buggy, stale docs)
`MockRerankerV1`	N/A	Deterministic score-sorted reranker
`MockGeneratorV1`	0.35 (stale) / 0.90 (fresh)	Detects stale context automatically
`MockMemoryReadV1`	N/A	Returns empty memory (safe default)
`MockMemoryWriteV1`	N/A	Disabled — never writes in prototype

`MockRetrieverV2Experimental` is the component intentionally designed to trigger the golden demo failure, producing `[STALE-2021]` and `[STALE-2020]` documents that cause the generator's faithfulness score to drop from 0.90 to 0.35.

`SandboxedWorker` — Multiprocessing Isolation (`sandbox.py`)

```
class SandboxedWorker:
    @staticmethod
    def run(func, inputs, timeout_seconds=5, trace_id="default"):
        manager = multiprocessing.Manager()
        return_dict = manager.dict()
        p = multiprocessing.Process(target=_sandboxed_execution_wrapper
                                    args=(func, inputs, return_dict, t
        p.start()
        p.join(timeout=timeout_seconds)
        if p.is_alive():
            p.terminate()
            raise TimeoutError("Sandboxed execution timed out.")
```

Isolation layers:

1. **OS-level process boundary** via `multiprocessing.Process`.
2. **Python audit hook** via `sys.addaudithook(_sandbox_audit_hook)` — blocks socket, file writes, subprocess.
3. **ARC Isolator** via monkey-patching — redirects network/shell to `HardwareDataSink`.

Staged actions: Blocked operations are **not silently dropped**. They are recorded as structured `staged_actions` dicts with `trace_id`, `type`, and `payload`. These are returned via the `Manager().dict()` shared memory and relayed to `VTICoordinator.stage_action()` in the parent process.

6.7 `policy` — Hierarchical Policy Engine

Files: `engine.py` (205 lines), `hierarchy.py` (786 lines), `resolver.py` (597 lines), `tiers.py` (474 lines), `approvals.py` (1193 lines)

Policy Hierarchy Architecture

```
Org Level          ← Highest precedence (defines the floor for all)
  └── Business Unit
```

```
        └── Pipeline
              └── Agent ← Most specific (can only tighten further)
```

Tightening-only rule (`resolver.py`):

- A child node can apply a **more restrictive** policy than its parent.
- A child node **cannot relax** a parent's deny rule.
- Attempting to relax without an explicit justification raises `PolicyConflictError`.
- This is deterministic and produces an `EffectivePolicy` audit record.

Risk Tier Registry (`tiers.py`)

Tier	Example Actions	Approval Requirements
LOW	Read metrics, get trace	None
MEDIUM	Config patch, route change	Team lead review
HIGH	Rollback, disable component	Manager + 1 additional approver
CRITICAL	Full system quarantine	Two-person control (2PC)

Approval Service (`approvals.py` — 1193 lines)

Complete approval workflow:

- `create_request(action, requester_id, justification)` — creates an `ApprovalRequest` with expiry.
- `approve(request_id, approver_id)` — validates not self-approval, records approval.
- `deny(request_id, approver_id, reason)` — records denial with mandatory reason.
- `expire(request_id)` — marks request as expired if not actioned within window.
- `break_glass(request_id, approver_id, emergency_reason)` — emergency bypass with mandatory audit event. Cannot be used without recording the emergency justification.
- `add_delegated_approver(request_id, delegate_id)` — adds a secondary approver.

Self-approval block: `if approver_id == request.requester_id: raise SelfApprovalError`.

Policy Engine Evaluation (`engine.py`)

```
decision = engine.evaluate(
    action="apply_rollback",
    tenant_id="acme",
    node_id="pipeline_v2",
    requester_id="user_alice",
    requester_role="operator"
)
# decision.verdict → "allow" | "deny" | "needs_approval"
# decision.rule_id → specific rule that fired
# decision.policy_version → traceable to exact policy revision
```

Default-deny contract (all cases explicitly handled):

- Unknown action → `DENY`
- Missing policy node → `DENY`
- Any resolver error → `DENY` (fail-closed)
- HIGH/CRITICAL without pending approval → `NEEDS_APPROVAL`

Shadow Mode (`shadow.py`)

Before deploying a new policy version:

1. Load the candidate policy.
2. Replay the last N historical policy evaluations.
3. Compare decisions: `{tightened: [...], relaxed: [...], unchanged: [...]}`.
4. If any `relaxed` decisions exist without justification, deployment is blocked.

Integration Hooks (`hooks.py`)

Every phase of the closed loop is gated:

```
pre_replay_check(action="run_replay", ...)      # Before replay executi
pre_recovery_check(action="apply_rollback", ...) # Before recovery acti
pre_execution_check(action="run_tool", ...)      # Before tool call exe
pre_rollback_check(action="rollback_version", .) # Before version rollb
```

Each raises `PolicyDeniedError` on deny verdict, preventing execution.

6.8 `recovery` — Recovery Orchestrator

Files: `engine.py`, `executor.py` (435 lines), `actions.py` (1021 lines), `capsule.py` (717 lines), `state_machine.py` (687 lines), `canary.py` (584 lines)

Saga State Machine (`state_machine.py`)

```
PENDING
  → PREPARE    (Create RollbackCapsule; snapshot current state)
  → EXECUTE    (Apply recovery action via executor)
  → VERIFY     (Run canary checks)
  → COMMITTED  (Success path)
  → COMPENSATING (Canary failed; executing capsule rollback)
  → COMPENSATED  (Rollback complete)
  → FAILED     (Unrecoverable error)
```

Each state transition is logged with `saga_id`, `action`, `timestamp`, and `operator`.

9 Allowlisted Recovery Actions (`actions.py`)

```
ACTION_REGISTRY = {
    RecoveryActionType.ROLLBACK_COMPONENT_VERSION: ActionSpec(
        required_params=["component_type", "target_version_id", "expect
        optional_params=["justification"],
        idempotency_key_fields=["component_type", "target_version_id"]
    ),
```

```
    RecoveryActionType.APPLY_CONFIG_PATCH: ActionSpec(...),
    RecoveryActionType.CHANGE_ROUTE_WEIGHT: ActionSpec(...),
    RecoveryActionType.DISABLE_COMPONENT: ActionSpec(...),
    RecoveryActionType.QUARANTINE_COMPONENT: ActionSpec(...),
    RecoveryActionType.RETRY_WITH_BACKOFF: ActionSpec(...),
    RecoveryActionType.TRIGGER_HUMAN_MUTATION: ActionSpec(...),
    RecoveryActionType.ENABLE_SHADOW_MODE: ActionSpec(...),
    RecoveryActionType.EMIT_ALERT: ActionSpec(...),   # Non-mutating
}
```

Any action not in this registry raises `ActionNotAllowedError`.

Recovery Executor Framework (`executor.py`)

Abstract `RecoveryExecutor` interface — every executor must:

1. Validate action type against `ACTION_REGISTRY` allowlist.
2. Check idempotency key — raise `IdempotencyConflictError` if already used.
3. Validate `expected_version_id` matches live component — raise `StaleVersionError` if stale.
4. Create `RollbackCapsule` with complete state snapshot **before** any mutation.
5. Execute action through the allowlisted adapter method.
6. Return `ExecutionResult(success, outcome, capsule_id, execution_mode)`.

`LocalDevExecutor` — in-memory simulator:

- No real service calls, no database writes.
- Deterministic and reproducible.
- Safe for tests and development.
- `execution_mode = ExecutionMode.DRY_RUN` by default.

Production executor injection:

- Real service adapters are **not imported by default**.
- They are injected via the executor factory only when `DRIFTGUARDX_ENV=production`.

Rollback Capsule (`capsule.py`)

```
@dataclass
class RollbackCapsule:
    capsule_id: str
    component_type: ComponentType
    target_version_id: UUID      # Version to rollback TO
    from_version_id: UUID        # Current (post-incident) version
    config_snapshot: dict        # Complete config at time of capsule c
    created_at: datetime
    expires_at: datetime         # Capsules expire to prevent stale rol
    is_applied: bool
    compatibility_constraints: List[CompatibilityConstraint]
```

`CompatibilityConstraint` checks:

- Minimum compatible version of dependent components.
- Config schema compatibility with the rollback target version.

Canary Verification (`canary.py`)

After every recovery action, bounded canary checks verify (within the `canary_window_seconds`):

- `quality_improvement > quality_threshold` (reliability score must improve)
- `p99_latency < latency_threshold_ms` (latency must not regress)
- `cost_delta_usd < cost_budget_usd` (cost must not exceed budget)
- Safety checks (no new policy violations in canary window)

If canary fails, the state machine transitions to `COMPENSATING` and automatically executes the capsule rollback.

6.9 `ledger` — Cryptographic Audit Ledger

Files: `chain.py` (156 lines), `schema.py` (272 lines), `crypto.py` (320 lines), `claims.py` (576 lines), `export.py` (287 lines)

Append-Only Chain (`chain.py`)

SQLite schema (upgradeable to PostgreSQL):

```
CREATE TABLE ledger (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    cert_id       TEXT UNIQUE NOT NULL,
    cert_hash     TEXT UNIQUE NOT NULL,   -- SHA-256 of canonical cert
    previous_hash TEXT NOT NULL,          -- Hash of previous record (c
    payload       JSON NOT NULL,          -- Full certificate JSON
    signature     TEXT NOT NULL,          -- Ed25519 signature (base64)
    signer_pub_key TEXT NOT NULL,         -- Public key for independent
    timestamp     TEXT NOT NULL           -- ISO-8601 UTC
)
```

`append_certificate()` protocol:

1. Validate `cert.previous_cert_hash == self._head_hash` (no forks).
2. Compute `cert_hash = SHA-256(cert.canonical_bytes())`.
3. Verify Ed25519 signature: `verify_signature(public_key, canonical_bytes, signature)`.
4. Execute `INSERT` atomically.
5. Update `_head_hash = cert_hash`.

`verify_chain()` — complete integrity check:

- Traverses all records in order from `GENESIS`.
- For each record: checks hash linkage AND re-verifies signature.
- Returns `False` on the first broken link or failed signature.

`RecoveryCertificate` Schema (`schema.py`)

Every certificate includes:

- `schema_version` — enables format evolution without breaking existing verifiers.
- `domain_separator` — prevents cross-context signature replay attacks.
- `canonical_bytes()` — deterministic UTF-8 JSON serialization (sorted keys) for signing.
- `compute_hash()` — SHA-256 of canonical bytes.

The `apps/cli/verifier.py` can verify an exported bundle without running the full stack, making it a portable compliance tool.

Cryptography (`crypto.py`)

- Ed25519 key generation: `generate_ed25519_keypair() → (private_pem, public_pem)`.
- `sign_certificate(private_key_pem, cert)` → base64 signature.
- `verify_signature(public_key_pem, data, signature)` → `bool`.
- KMS stub: `sign_with_kms(cert)` — placeholder for AWS KMS / Azure Key Vault.

6.10 `evaluation` — Benchmarking & Experiment Tracking

Key modules:

`reliability.py`:

```
reliability_vector = compute_reliability_vector(span_records)
# Returns: {"faithfulness": 0.85, "latency": 0.72, "safety": 1.0, "cost

delta = compute_reliability_delta(original_vector, replay_vector)
# Returns: {"faithfulness": +0.55, "latency": +0.0, "safety": 0.0, "cos

score = aggregate_reliability_score(vector, weights)
# Returns: 0.87 (weighted sum)
```

`bandit_baselines.py` — three comparison baselines for the BCRB:

- `RandomBudgetScheduler` — random arm selection within budget.
- `CheapestFirstScheduler` — always picks the cheapest eligible arm.
- `GreedyPriorScheduler` — picks the arm with highest prior.

`analysis/stats.py` — publication-grade statistical tools:

- Paired bootstrap resampling with configurable B (bootstrap iterations).
- Permutation test for significance.
- Cohen's d effect size computation.
- Bonferroni correction for multiple comparisons.
- Outputs formal `docs/statistical_report.md`.

`experiments/orchestrator.py`:

- Routes fault regimes across dataset slices.
- Runs detector-only, bcrb, exhaustive-replay, and combined evaluation configs.
- Records all metrics to MLflow.

6.11 `rationale` — Explainability & LLM Rationale

`RationaleInputContract` — strict Pydantic schema bounding evidence:

- Only pre-computed, measured values from `ReplayEpisode` and `Diagnosis` can be passed.
- Prevents the LLM from inventing supporting evidence.

`templates.py` — deterministic generation without LLM:

- Four output formats: Operator (technical), Executive (business), Incident Report, Patent Evidence.
- Guaranteed to never hallucinate — pure string templating from validated input.

`validator.py` — hallucination scanner:

- Checks that all component names in the rationale appear in the evidence input.
- Checks that all version tags in the rationale match the input version tags.
- Checks that all numeric metrics in the rationale match computed metrics (within ± 0.01 tolerance).
- Returns `False` if any invented content is detected; triggers fallback to templates.

`llm.py` — optional LLM enhancement:

- Passes evidence to LLM with strict prompt constraints.
- Auto-redacts any PII from trace data before LLM submission.
- Runs hallucination validator on LLM output.
- Falls back to template instantly if validation fails.

---

7. Applications Layer

7.1 API — FastAPI Backend

Location: `apps/api/src/`

11 SQLAlchemy ORM tables (`models.py` — 361 lines):

Table	Key Columns	Notes
`tenants`	id, name, slug, is_active	Multi-tenant root. Slug indexed.
`audit_events`	tenant_id, user_id, action, resource_type	Append-only security log
`agent_pipelines`	tenant_id, name, version, component_versions (JSONB)	Versioned pipeline catalog
`component_versions`	component_type, version_tag, state, config_hash, parent_version_id	Version lineage tree
`request_runs`	tenant_id, pipeline_id, status, reliability_score, reliability_vector (JSONB)	Execution records

`span_records`	trace_id, span_id, input_hash, output_hash, latency_ms	Hash-only span storage
`trace_artifacts`	run_id, spans (JSONB), root_span_id	Normalized trace container
`replay_episodes`	run_id, swapped_component, reliability_delta, pinned_version_ids (JSONB)	Counterfactual results
`diagnoses`	run_id, root_cause_component, claims (JSONB)	Causal diagnosis
`repair_decisions`	diagnosis_id, status, proposed_intervention, approved_by	Human approval records
`recovery_certificates`	run_id, replay_episode_id, certificate_hash, is_valid	Crypto certificates

Dual-backend support:

```
if "postgresql" in _DB_URL:
    from sqlalchemy.dialects.postgresql import JSONB as _JSON_TYPE
else:
    from sqlalchemy import JSON as _JSON_TYPE
```

Production uses PostgreSQL JSONB (indexed, queryable). Development falls back to SQLite JSON.

Database indexes:

- `tenants.slug` — unique lookup.
- `audit_events.tenant_id` — tenant isolation.
- `request_runs.(tenant_id, pipeline_id)` — composite.
- `span_records.(tenant_id, run_id)` — trace retrieval.
- `component_versions.(tenant_id, component_type)` — version lookup.

7.2 Worker — Background Job Engine

The worker uses **ARQ** (Asyncio Redis Queue) for:

Job	Estimated Duration	Priority
`build_causal_graph`	0.1–2s	High (blocks diffusion)
`run_diffusion_inference`	1–10s	High (blocks replay)
`execute_replay`	1–30s	Medium
`sign_certificate`	10–100ms	High (blocks ledger append)
`verify_ledger_chain`	1–60s	Low (scheduled)

7.3 Web Console — Next.js 14 Dashboard

14 pages with Truthful UI:

Every data value displays its epistemological status:

- 🌐 `MEASURED` — computed from real data.

- 🔘 `INFERRED` — derived from model predictions.
- 🔘 `SYNTHETIC` — generated from test data.
- 🔘 `CERTIFIED` — passed all bounds checks.
- 🔘 `UNCERTIFIED` — bounds checks not yet run.
- ○ `UNAVAILABLE` — data not yet available.

The UI never displays a confidence percentage without showing its bound method and observed coverage alongside it.

7.4 CLI — Developer Tooling

`verifier.py` — portable, standalone ledger verifier:

- Reads an exported bundle JSON (from `ledger/export.py`).
- Recomputes all certificate hashes.
- Re-verifies all Ed25519 signatures.
- Reports chain integrity without requiring database access.

`experiments.py` — evaluation runner:

- `python -m apps.cli.experiments run --config ablation_v1 --faults retriever_stale`
- Outputs MLflow run ID and experiment metrics.

---

8. The Closed-Loop Pipeline

Scenario: RAG Agent Returns Hallucinated Answer

Step 1 — Tracing: Agent executes. TraceContext wraps each span. Input/output hashed. Spans submitted via `POST /spans/batch`. No raw prompts stored.

Step 2 — Graph Build (Worker):

```
event:retriever → version:v2-exp          [VERSION_LINEAGE]
event:retriever → event:reranker          [CONTROL_FLOW]
event:reranker  → event:generator         [CONTROL_FLOW]
memory:m_001    → event:generator         [MEMORY_INFLUENCE]
```

Step 3 — Drift Detection:

- `ks_test` on retrieval scores: $p = 0.003$ (significant shift detected).
- `FaithfulnessDetector`: faithfulness_hint = 0.35 vs baseline 0.90.
- `SymptomRegistry.register("faithfulness_drift", severity=HIGH)`.

Step 4 — GAT Diffusion:

- `root_pred[retriever] = 0.89` → root cause.
- `symptom_pred[generator] = 0.76` → symptom.
- No MEMORY→TOOL edge detected; cybersecurity firewall not tripped.

Step 5 — BCRB Selection:

- Budget: 10 tokens.

- `arm_a: rollback_retriever_v2→v1 (cost=4, prior=0.8)` → selected.
- `arm_b: config_patch (cost=8)` → survivable.
- `arm_c: disable_generator (cost=6, history shows cost≈15)` → PREEMPTIVELY SHED.

Step 6 — Counterfactual Replay:

- Pins all components. Swaps retriever v2-exp → v1.
- MockRetrieverV1 returns fresh docs (faithfulness = 0.90).
- `reliability_delta = +0.62` confirmed.

Step 7 — Policy Gate:

- Risk tier: HIGH.
- `verdict = needs_approval`.
- Manager approval request created.

Step 8 — Human Approval:

- Manager approves via Web Console.
- `RepairDecision.status = APPROVED`.

Step 9 — Recovery:

- PREPARE → `RollbackCapsule` created with v2-exp config snapshot.
- EXECUTE → `LocalDevExecutor` swaps to v1.
- VERIFY → `CanaryVerifier` confirms +0.62 improvement.
- COMMITTED.

Step 10 — Certificate & Ledger:

- `RecoveryCertificate` issued, Ed25519-signed.
- `LedgerChain.append_certificate()` validates and commits.
- Web Console `/ledger` shows new entry with green chain integrity status.

---

9. Agent Architecture

9.1 How Many Agents Does DriftGuard-X Support?

DriftGuard-X supports **unlimited heterogeneous agents** simultaneously. Each agent registers itself in the causal graph as a `NodeType.AGENT` node.

Agent archetype mapping:

Archetype	ComponentType	Graph Representation
Researcher / Retriever Agent	RETRIEVER → AGENT	Retrieves information, synthesizes context
Executor Agent	TOOL_CALL → AGENT	Takes actions (SQL writes, API calls, emails)
Generator / LLM Agent	GENERATOR → AGENT	Produces natural language outputs

Memory Agent	MEMORY_READ / MEMORY_WRITE → AGENT	Long-term state management
Validator / Safety Agent	POLICY_CHECK → AGENT	Output quality and safety checking
Coordinator Agent	AGENT (root)	Orchestrates sub-agents

9.2 Inter-Agent Communication Edges

Attribute-based registration: Any span can record:

```
span.attributes["dgx.agent.message_to"] = "executor_agent_001"
```

The `GraphBuilder` automatically creates:

1. `GraphNode(id="agent:executor_agent_001", type=NodeType.AGENT)`
2. `GraphEdge(type=EdgeType.INTER_AGENT_COMMUNICATION, source="event:span_id", target="agent:executor_agent_001")`

Multi-agent pipeline example:

```
agent:researcher  ──[INTER_AGENT_COMMUNICATION]──► agent:executor
agent:executor    ──[CONTROL_FLOW]──► event:tool_call
event:tool_call   ──[VERSION_LINEAGE]──► version:sql_api_v1
```

9.3 Blame Diffusion in Multi-Agent Systems

When the Researcher Agent hallucinates:

1. `root_pred[researcher] = 0.87` (GAT identifies Researcher as root cause)
2. The INTER_AGENT_COMMUNICATION edge propagates anomaly signal.
3. `symptom_pred[executor] = 0.65` (Executor is a symptom, not the root)
4. `symptom_pred[tool_call] = 0.71` (Tool call is downstream of the poisoned message)

The system correctly attributes blame to the Researcher, not the Executor, even though the visible failure manifested at the tool call level. Without INTER_AGENT_COMMUNICATION edges, the Executor would appear to be the root cause.

The `loss_contrastive` term specifically trains the model to handle this case: for nodes where `root_true=0, symptom_true=1` (connected downstream via inter-agent message), the model is penalized for confusing root with symptom.

---

10. Advanced Subsystems (Session Additions)

10.1 VTI — Two-Phase Commit Coordinator

File: `packages/replay/src/vti_coordinator.py` — 80 lines

`CryptographicEscrow`:

- `payload_hash = SHA-256(json.dumps(payload, sort_keys=True))` — computed at escrow creation.

- Status: `STAGED → COMMITTED | ROLLED_BACK`.

Phase 1 — Stage:

```
escrow = vti.stage_action(trace_id, "SQL_UPDATE", {"table": "users", "s
# escrow.status = "STAGED"
# escrow.payload_hash = "a3f2b1..."
# No real-world effect.
```

Phase 2 — Commit (requires GAT clearance):

```
vti.commit_action(trace_id, "GAT-CLEAR-550e8400-e29b-...")
# Validates signature format
# escrow.status = "COMMITTED"
```

Phase 2 — Rollback (on drift detection):

```
vti.rollback_action(trace_id)
# escrow.status = "ROLLED_BACK"
# No real-world effect; escrow discarded
```

Innovation: This is the first application of a Two-Phase Commit protocol to AI agent action authorization, where the commit authorization token is a GNN-generated cryptographic clearance signature.

10.2 ARC Isolator — Physical Execution Channel Isolation

File: `packages/replay/src/arc_isolator.py` — 130 lines

`HardwareDataSink` — thread-safe quarantine:

```
data_sink.commit("NETWORK_CALL", {"event": "socket.connect", "address":
data_sink.commit("SHELL_EXEC",   {"event": "os.system",     "command":
quarantined = data_sink.get_all()  # Forensic analysis after the run
```

`MockSocket` — transparent loopback:

- `connect(address)` → logs to `HardwareDataSink`, returns immediately.
- `send(data)` → logs to `HardwareDataSink`, returns `len(data)`.
- `recv(bufsize)` → returns `b"HTTP/1.1 200 OK\r\n\r\n{\"mock\": \"arc_isolated_response\"}"`.

`ARCIsolator.enable()` — monkey-patches Python runtime:

```
os.system   = mock_system    # → data_sink
subprocess.run = mock_run    # → data_sink
socket.socket  = mock_socket_init  # → MockSocket
```

`ARCIsolator.disable()` — restores originals via `finally` block.

Why not just use audit hooks? Python `sys.addaudithook` can only raise exceptions, not return mock values. Monkey-patching at the Python object layer allows transparent interception.

10.3 AOR Scheduler — Asynchronous Optimizer Recomputing

File: `packages/replay/src/aor_scheduler.py` — 110 lines

Task lifecycle:

```
PENDING → RUNNING → COMPLETED
                ↓
            FAILED → DIAGNOSING → FAILED (with diagnostic_result)
                ↓
        (dependents) → BLOCKED
```

Thread pools:

- `_executor` (max_workers = 4): Primary compute pool.
- `_diagnostic_executor` (max_workers = 2): VTI sandbox diagnostic pool.

Scheduling loop (polling with 100ms timeout):

```
while not all_done:
    ready_tasks = [t for t in tasks if all deps COMPLETED]
    for task in ready_tasks:
        task.status = RUNNING
        _executor.submit(_execute_task, task)
    _condition.wait(timeout=0.1)
```

Failure handling:

```
def _execute_task(task):
    try:
        result = task.func(**task.inputs)
        task.status = COMPLETED
    except Exception as e:
        task.error = e
        task.status = DIAGNOSING
        _diagnostic_executor.submit(_run_diagnostic, task)
        # Primary pool is immediately freed for other tasks
```

The key innovation: Failed tasks are diverted to a **secondary diagnostic pool** while the primary pool is immediately freed to process independent tasks. This prevents the N-1 CPU waste caused by a single agent failure.

10.4 JIT Graph Hydration — RAM Optimization

File: `packages/replay/src/jit_hydration.py` — 100 lines

`LazyStateStore` API:

```
store = LazyStateStore({"var_a": huge_tensor, "var_b": large_doc, ...,
store.hydrated_count   # → 0 (nothing loaded yet)
store.total_variables  # → 26

val = store.hydrate("var_a")  # Load only var_a
store.hydrated_count           # → 1
```

BFS neighbourhood:

```
hydrator = JITGraphHydrator(
    graph={"node_a": ["node_b", "node_c"], "node_b": ["node_d"], ...},
    state_store=store
)
result = hydrator.hydrate_for_node("node_a", depth=1)
# Hydrates: {node_a, node_b, node_c} → 3 variables
# Leaves {node_d, node_e, ..., node_z} unhydrated → N-3 variables saved
```

Algorithm (correct BFS):

```
visited = {failing_node}
frontier = {failing_node}
for _ in range(depth):
    next_frontier = set()
    for node in frontier:
        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                visited.add(neighbour)
                next_frontier.add(neighbour)
    frontier = next_frontier
return visited
```

Patent claim: RAM footprint reduced from O(N) to O(K) where K = neighbourhood size ≤ degree(failing_node) × depth.

10.5 Semantic Circuit Breaker — Network I/O Optimization

File: `packages/replay/src/semantic_circuit_breaker.py` — 140 lines

15 mutating keywords:

```
_MUTATING_KEYWORDS = [
    "UPDATE", "DELETE", "INSERT", "DROP", "TRUNCATE", "ALTER",  # SQL
    "POST", "PUT", "PATCH",                                      # HTTP
    "send_email", "write_file", "execute_sql", "publish", "commit"  # A
]
```

Two-pass inspection:

```
def inspect(self, code_or_description: str) -> bool:
    # Pass 1: O(N) regex scan (very cheap)
    trigger = self._detect_via_keywords(code_or_description)

    # Pass 2: AST parse (if valid Python)
    if trigger is None:
        trigger = self._detect_via_ast(code_or_description)

    if trigger:
        self.state = CircuitState.TRIPPED
        self.trip_log.append({"trigger": trigger, "snippet": code[:200]
        return True
    return False
```

AST detection catches:

- `db.delete(record_id)` — method call on object.
- `requests.post(url, json=data)` — library function call.
- `execute_sql("DELETE FROM ...")` — wrapped function.

`execute_with_breaker()` — main gateway:

```
result = scb.execute_with_breaker("UPDATE accounts SET balance=0", real
# Circuit TRIPPED → returns synthetic payload, real_db_call never calle
# Zero HTTP bytes serialized, zero network I/O
```

10.6 Merkle-DAG State Deduplication — Storage Optimization

File: `packages/replay/src/merkle_dag.py` — 140 lines

Hash computation:

```
def _content_hash(payload):
    return sha256(json.dumps(payload, sort_keys=True).encode()).hexdige

# MerkleNode hash:
raw = _content_hash(payload) + "".join(sorted(parent_hashes))
node.node_hash = sha256(raw.encode()).hexdigest()
```

Deduplication example:

```
store = MerkleDAGStore()

# 1000 failure traces, all using the same RAG prompt
for i in range(1000):
    store.add_node(f"trace_{i}/span_0", {"prompt": "What is capital of

print(store.blob_count)  # → 1 (one unique blob)
print(store.node_count)  # → 1 (all 1000 nodes deduplicated to 1)
```

Tamper detection:

```
store.add_node("audit_record", {"decision": "approve", "amount": 50000}
store._nodes["audit_record"].payload = {"decision": "approve", "amount"
store.verify_chain("audit_record")  # → False (tamper detected)
```

10.7 Pre-emptive Compute Shedding — GPU Optimization

Integrated into: `packages/replay/src/bandit.py`

Confidence model:

```
def _overbudget_confidence(self, arm):
    history = self._cost_history.get(arm.arm_id, [])
    if len(history) < 2:
        return 0.0          # Optimistic: no shedding without evidence

    mean = statistics.mean(history)
    std  = statistics.stdev(history)
    z = (mean - self.remaining_budget) / std
```

```
    confidence = 1.0 / (1.0 + exp(-1.7 * z))  # Logistic CDF approximat
    return confidence
```

99% threshold: `if confidence >= 0.99: shed_log.append(arm.arm_id); return True`

Example:

```
sched = BCRBScheduler(total_budget=5.0)
sched._cost_history["arm_b"] = [50.0, 51.0, 52.0, 53.0, 54.0]
# mean=52, std≈1.5, z=(52-5)/1.5=31.3 → confidence≈1.0 → SHED
sched.select_arm([arm_a, arm_b])  # arm_b shed; arm_a selected
```

---

11. Security Architecture

Threat Model Summary (STRIDE)

Threat	Mitigation	Implementation
Spoofing (fake tenant)	JWT `tenant_id` claim extraction	`auth.py` + RLS on all queries
Tampering (ledger manipulation)	Ed25519 hash chain	`chain.py verify_chain()`
Repudiation (deny action)	Append-only audit log	`AuditEventORM` + ledger
Information Disclosure (PII leak)	Hash-only storage + redaction	`tracer.py redact_dict()`
Denial of Service	Timeout + budget limits	`SandboxedWorker` timeout, BCRB budget
Elevation of Privilege	Default-deny policy engine	`PolicyEngine` + `InheritanceResolver`

Security Test Coverage (`tests/security/test_policy_security.py`)

15 security tests including:

- Cross-tenant isolation (tenant A cannot read tenant B's data).
- Confused deputy (intermediate service cannot escalate privileges).
- Self-approval block (requester cannot approve own request).
- Break-glass audit trail (emergency bypass recorded with mandatory justification).
- Policy determinism (same input always produces same verdict).
- Tightening inheritance (child policy cannot relax parent's deny rule).

Sandbox Security Layers (Defense-in-Depth)

```
Layer 1: OS process boundary (multiprocessing.Process)
Layer 2: Python audit hooks (sys.addaudithook)
         → Blocks: socket, open-write, subprocess
Layer 3: ARC Isolator (monkey-patching)
         → Redirects: socket.socket, os.system, subprocess.run
Layer 4: Semantic Circuit Breaker
```

```
        → Pre-empts: AST-detected mutating intent
```

Each layer provides independent protection, making bypass increasingly difficult.

---

12. Test Coverage & Quality

Test Suite Breakdown

Category	File Count	Approximate Tests	Focus
E2E	25	~120	Full pipeline integration
Security	2	15+	Attack boundary enforcement
Integration	Multiple	Variable	DB + Redis
Contract	Multiple	Per-model	Pydantic validation
Unit	Multiple	Extensive	Per-function
Total	**~40+**	**156+**	

E2E Test Inventory (25 files)

File	Tests	Critical Scenarios
`test_golden_demo.py`	5	Full closed-loop replay → certificate
`test_recovery.py`	15+	All 5 failure modes including saga compensation
`test_ledger_tamper.py`	10+	Hash tampering, signature forgery, fork detection
`test_four_enhancements.py`	**21**	All 4 optimization enhancements
`test_arc_isolator.py`	3	Physical isolation + loopback verification
`test_aor_scheduler.py`	1	Multi-agent AOR + VTI diagnostic
`test_vti_2pc.py`	5+	2PC stage, commit, rollback, invalid signature
`test_firewall_signatures.py`	3+	GAT MEMORY→TOOL signature detection
`test_multi_agent_diffusion.py`	4	INTER_AGENT_COMMUNICATION edge diffusion
`test_bounds_calibration.py`	10+	Statistical bounds + calibration validation
`test_security.py`	5+	Prompt injection, malicious outputs, spoofing
`test_chaos.py`	5+	Worker failover, Redis failover, DB timeout
`test_load.py`	3+	Concurrent load, TPS baseline
`test_rationale_eval.py`	8	Hallucination detection, fallback verification
`test_bcrb_scheduler.py`	5+	BCRB selection, budget exhaustion
`test_graph_properties.py`	4	DAG validation, edge types

Quality Gates (`pyproject.toml`)

Ruff rules enforced:

- `E, F, W` — PEP 8 style and warnings.

- `I` — Import ordering.
- `N` — Naming conventions.
- `UP` — Python upgrade opportunities.
- `B` — Bug-prone patterns.
- `SIM` — Simplification opportunities.
- `TCH` — Type-checking imports.
- `ANN` — Missing type annotations (relaxed for tests/examples).
- `S` — Security issues.
- `RUF` — Ruff-specific rules.

MyPy strict mode:

- `strict = true`
- `warn_return_any = true`
- `warn_unused_configs = true`
- `ignore_missing_imports = false`
- `plugins = ["pydantic.mypy"]`

---

13. Observability & Telemetry

OpenTelemetry Integration

`infra/otel-collector-config.yaml`:

- OTLP receiver on ports 4317 (gRPC) and 4318 (HTTP).
- Prometheus exporter on port 8888.
- Configurable exporters for Jaeger, Zipkin, or commercial backends.

DGX Span Attributes:

```
dgx.tenant_id              → Multi-tenant isolation key
dgx.pipeline_id            → Pipeline version UUID
dgx.component_type         → ComponentType enum value
dgx.component_version_id   → Exact version UUID
dgx.component_version_tag  → Human-readable version tag
dgx.memory.referenced      → Memory node ID (for MEMORY_INFLUENCE edges
dgx.agent.message_to       → Target agent ID (for INTER_AGENT_COMMUNICA
dgx.policy_result          → "allow" | "deny" | "needs_approval"
dgx.replay.episode_id      → Links span to its replay episode
dgx.certificate.id         → Links span to its ledger certificate
```

Structured Logging with structlog

Every log event is a JSON object with:

```
{
  "event": "policy_decision",
  "tenant_id": "acme",
  "run_id": "...",
  "action": "apply_rollback",
  "verdict": "needs_approval",
  "rule_id": "HIGH_RISK_DEFAULT",
```

```
  "policy_version": "v3.2.1",
  "timestamp": "2026-07-25T07:35:22.123Z"
}
```

MLflow Experiment Tracking

All evaluation runs tracked with:

- **Parameters:** fault type, detector config, bandit budget, model version.
- **Metrics:** reliability_delta mean/variance, BCRB efficiency, detector precision/recall, FPR.
- **Artifacts:** confusion matrices, efficiency frontier plots, calibration curves.
- **Tags:** git_sha, environment, seed.

---

14. Data Models & Database Schema

Primary Database Table Summary

Table	Estimated Rows/ Day	Growth Pattern	Partition Strategy
`tenants`	1-10	Slow	None needed
`audit_events`	100-10000	High	By `created_at` monthly
`request_runs`	1000-100000	High	By `tenant_id` + `created_at`
`span_records`	10x runs	Very High	By `run_id` or `created_at`
`trace_artifacts`	Same as runs	High	By `run_id`
`replay_episodes`	~10% of runs	Medium	By `run_id`
`recovery_certificates`	~1% of runs	Low	None needed
`ledger` (SQLite)	Same as certs	Low	Archive old entries

Key Database Indexes

```
-- Request runs (most queried)
CREATE INDEX ix_request_runs_tenant_pipeline ON request_runs(tenant_id,
CREATE INDEX ix_request_runs_created_at ON request_runs(created_at DESC

-- Spans (high write, high read volume)
CREATE INDEX ix_span_records_trace ON span_records(trace_id, span_id);
CREATE INDEX ix_span_records_tenant_run ON span_records(tenant_id, run_

-- Component versions (join-heavy)
CREATE INDEX ix_component_versions_type ON component_versions(tenant_id
```

---

15. Scalability Architecture

Current Single-Node Limits (Prototype)

Metric	Current Limit	Bottleneck
Span ingestion TPS	~500	Single SQLite in dev
Graph builds/min	~60	Single worker process
GAT inference/min	~30	Single CPU (no GPU in dev)
Concurrent replays	4 (ThreadPool)	`max_workers` setting
Ledger appends/sec	~100	aiosqlite WAL mode

Horizontal Scaling Plan

API tier → Kubernetes Deployment:

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: api
        resources:
          requests: {cpu: "500m", memory: "512Mi"}
          limits:   {cpu: "2000m", memory: "2Gi"}
```

HPA on CPU utilization $>$ 70%.

Worker tier → Dedicated node pools:

- Graph build workers: 2+ CPU-heavy pods.
- GAT inference workers: 1-2 GPU-equipped pods (V100 or A100).
- Certificate signing workers: 1-2 lightweight pods.

Database tier:

- PostgreSQL → AWS RDS Multi-AZ with read replicas.
- Connection pooling via PgBouncer.
- Citus extension for `tenant_id`-based sharding at high scale.

AOR Scheduler at scale:

- Replace `ThreadPoolExecutor` with Kubernetes Job dispatcher.
- Each `AORTask` → K8s Job with `backoffLimit=0` and `restartPolicy=Never`.
- Failed tasks → diagnostic K8s Jobs in isolated namespaces with network policies.

Merkle-DAG deduplication at scale:

- Blob store → AWS S3 or Google Cloud Storage (content-addressed, immutable).
- Node index → Redis Sorted Set (node_hash → node_id).
- DAG traversal → dedicated graph database (Amazon Neptune or Neo4j).

GAT Model serving at scale:

- ONNX export for platform-agnostic inference.
- Triton Inference Server for GPU-batched inference.

- Model versioning via MLflow Model Registry.

---

16. Patent & Research Claims Mapping

Claim	File	Evidence
VTI 2PC with GNN clearance token	`vti_coordinator.py`	`test_vti_2pc.py`
ARC physical channel isolation	`arc_isolator.py`	`test_arc_isolator.py`
AOR asynchronous CPU re-allocation	`aor_scheduler.py`	`test_aor_scheduler.py`
JIT graph hydration RAM reduction	`jit_hydration.py`	`test_four_enhancements.py::TestJI`
Semantic circuit breaker AST intent	`semantic_circuit_breaker.py`	`test_four_enhancements.py::TestSe`
Merkle-DAG trace deduplication	`merkle_dag.py`	`test_four_enhancements.py::TestMe`
Pre-emptive compute shedding	`bandit.py`	`test_four_enhancements.py::TestPi`
GAT cybersecurity firewall loss	`trainer.py`	`test_firewall_signatures.py`
Inter-agent blame diffusion	`builder.py` + `dataset.py`	`test_multi_agent_diffusion.py`
Tightening-only policy inheritance	`resolver.py`	`test_policy_security.py`
Ed25519 hash-chain ledger	`chain.py` + `crypto.py`	`test_ledger_tamper.py`
Version-pinned deterministic replay	`engine.py`	`test_golden_demo.py`

Budget-constrained Knapsack-UCB	`bandit.py` + `bandit_baselines.py`	`test_bcrb_scheduler.py` + `patent_e`
Hoeffding statistical bounds	`evaluation/analysis/stats.py`	`proof_appendix_bounds.md`

---

17. Deployment Readiness Audit

Green Light (Production-Ready Components)

Component	Readiness	Notes
Pydantic v2 data contracts	✓ Production-grade	617 lines, strict mode, tested
Ed25519 cryptographic signing	✓ Production-grade	Full verify + chain tests
Append-only ledger	✓ Production-grade	Tamper detection tested
Policy engine (default-deny)	✓ Production-grade	Hierarchical, 2-person control
Statistical bounds & calibration	✓ Publication-grade	Bootstrap, Bonferroni, Hoeffding
Docker Compose stack	✓ Functional	5 services with healthchecks
Alembic migrations	✓ Defined	Schema version-controlled
Multi-tenant isolation	✓ Implemented	tenant_id on all queries
PII redaction	✓ Implemented	Key + regex patterns
Test suite (156+ tests)	✓ Comprehensive	Chaos, security, load included

Yellow Light (Needs Work Before Production)

Component	Current State	Production Gap
Authentication	Mock OIDC JWT	Real OIDC provider (Auth0, Okta)
KMS signing	Python stub	AWS KMS / Azure Key Vault
Ledger backend	SQLite	PostgreSQL migration
Recovery adapters	In-memory mock	Real Kubernetes/Helm/FF adapters
ARC isolation	Python-level	Linux network namespace + iptables
GAT model	Demo data	Training on production traces
Rate limiting	Not implemented	API gateway or Nginx
Secret management	.env file	HashiCorp Vault
Backup/restore	Not implemented	Automated backup pipeline
Alerting	Not implemented	PagerDuty/OpsGenie integration

---

18. What Needs to Be Done for Production Deployment

Phase 1 — Infrastructure Hardening (Weeks 1-2)

1. Real OIDC Integration (Est: 3 days)

- Replace `apps/api/src/auth.py` mock tokens with Auth0, Okta, or Keycloak.
- Map OIDC claims to `tenant_id` and RBAC roles.
- Validate JWT expiry, issuer, and audience on every request.

2. Secret Management (Est: 2 days)

- Provision HashiCorp Vault or AWS Secrets Manager.
- Rotate the Ed25519 signing key with formal key ceremony.
- Remove all secrets from `.env` file; inject from Vault at startup.

3. KMS Integration (Est: 2 days)

- Replace `sign_with_kms()` stub in `packages/ledger/src/crypto.py`.
- Implement AWS KMS `sign` API call with `RSASSA_PSS_SHA_512` or Ed25519 CMK.
- Add key rotation policy and audit log monitoring.

4. Rate Limiting (Est: 1 day)

- Add Nginx or AWS API Gateway rate limiting.
- Per-tenant limits: 1000 spans/minute, 100 replays/minute.
- Return `429 Too Many Requests` with `Retry-After` header.

5. PostgreSQL Migration for Ledger (Est: 1 day)

- Migrate `packages/ledger/src/chain.py` from `aiosqlite` to `asyncpg`.
- Enable PostgreSQL native `APPEND-ONLY` enforcement via trigger.
- Run load test on PostgreSQL ledger at 1000 certs/second.

6. Backup & Restore (Est: 2 days)

- Automated nightly PostgreSQL dump to S3.
- Point-in-time recovery (PITR) enabled.
- Test restore procedure; run `verify_chain()` after every restore.

Phase 2 — ML Model Production Pipeline (Weeks 2-4)

7. GAT Model Training (Est: 2 weeks)

- Deploy trace SDK in shadow mode to production AI agents.
- Collect 30-90 days of real execution traces.
- Label root causes (from incident reports and human review).
- Train GAT with production graph structures.
- Validate: AUC-ROC $>$ 0.90 on held-out test set.

8. Model Registry & Versioning (Est: 3 days)

- Register trained GAT in MLflow Model Registry.
- Tag with `production` stage after validation.
- Implement A/B testing between model versions in production.

9. Model Drift Monitoring (Est: 2 days)

- Monitor GAT's own prediction distribution over time.
- Alert when `mean(root_pred) > 0.5` for more than N% of recent traces (model drift).

10. Calibration for Production (Est: 1 week)

- Run `packages/detectors/src/calibration.py` on 90-day trace history.
- Validate FPR $<$ 5% on held-out traces.
- Schedule weekly recalibration job.

Phase 3 — Real Recovery Adapters (Weeks 3-5)

11. Kubernetes Rollback Adapter (Est: 3 days)

```
class KubernetesRollbackExecutor(RecoveryExecutor):
    def execute(self, proposal):
        # kubectl rollout undo deployment/{component} --to-revision={ta
        self.k8s_client.rollout_undo(...)
```

12. Feature Flag Adapter (Est: 2 days)

```
class LaunchDarklyDisableAdapter(RecoveryExecutor):
    def execute(self, proposal):
        self.ld_client.toggle(proposal.params["flag_key"], enabled=Fals
```

13. Traffic Routing Adapter (Est: 2 days)

- Istio VirtualService weight patching.
- Kong route weight update.
- AWS ALB target group weight adjustment.

14. Canary Metrics Integration (Est: 3 days)

- Connect `CanaryVerifier` to Datadog metrics API.
- Pull P99 latency and error rate from real production metrics.
- Set SLO thresholds: P99 $<$ 500ms, error rate $<$ 0.1%.

Phase 4 — Observability & Operations (Weeks 4-6)

15. Production Alerting (Est: 2 days)

- OTel Collector → Datadog/Grafana Cloud.
- PagerDuty integration for `CRITICAL` drift events.
- SLO burn rate alerts (30-minute and 6-hour windows).

16. Load Testing at Scale (Est: 1 week)

- Run `test_load.py` at 10x, 100x concurrent load.
- Identify bottlenecks (expected: graph build at O(N spans)).
- Set hard concurrency limits and return 503 gracefully when overloaded.

17. Chaos Engineering (Est: 3 days)

- Run `test_chaos.py` against production stack.
- Simulate PostgreSQL failover → verify `LedgerChain` resumes correctly.
- Simulate Redis restart → verify ARQ worker reconnects and re-processes.

Phase 5 — Compliance & Legal (Weeks 5-8)

18. GDPR Right to Erasure (Est: 1 week)

- Implement `DELETE /tenants/{id}/data` endpoint.
- Verify erasure removes all traces, spans, runs, diagnoses.
- The ledger certificates cannot be deleted (by design); verify this is GDPR-compliant.

19. Patent Filing (Est: 2-4 weeks with counsel)

- Use `docs/patent_evidence_matrix.md` and `docs/patent_technical_disclosure.md`.
- File claims for 12+ novel mechanisms.
- Priority filing date: current RC1 release date.

20. SOC 2 Type II Preparation (Est: 3-4 weeks)

- Map `LedgerChain` to CC7.1 (Logical and Physical Access Controls).
- Map `PolicyEngine` to CC6.3 (Authorization).
- Map `AuditEventORM` to CC7.2 (System Operations Monitoring).
- Document key management lifecycle for CC6.7.

---

19. Known Limitations & Honest Constraints

From `LIMITATIONS.md` (Verbatim + Technical Depth)

1. Causal Diagnosis is Correlational Until Replayed

The `CausalGraph` identifies which component version was active during a failure. This is correlational evidence. `ReplayEpisode` upgrades this to interventional evidence. However, if external providers change their behavior between the original run and the replay (e.g., OpenAI's internal model updates), the reliability delta may be incorrectly attributed. This confounder is documented in every `ReplayEpisode.limitations` field.

2. Statistical Bounds Require i.i.d. Samples

The Hoeffding bounds assume i.i.d. samples. Production traffic exhibits:

- Temporal autocorrelation (peak hours vs off-hours).
- Seasonal distribution shifts.
- Concept drift (the correct answer changes over time).

The `observed_coverage` field always reports empirical coverage alongside the nominal confidence level, providing an honest check.

3. Privacy Mode Restricts Detector Accuracy

When `privacy_mode = METADATA_ONLY`:

- `FaithfulnessDetector` cannot run (no content available).
- Only latency, token count, and cost metrics are analyzable.
- This is a fundamental privacy-vs-observability tradeoff documented in the system.

4. Production Adapters Are Mocked

The `LocalDevExecutor` is entirely in-memory. No real service calls are made. This is the

most significant gap for production deployment.

5. Reproducibility Has a Time Horizon

Black-box APIs (OpenAI, Anthropic, Cohere) do not guarantee bit-for-bit reproducibility. A trace replayed 6 months later may produce different outputs even with the same seed and version vector.

6. ARC Isolator is Python-Level Only

A sufficiently motivated agent with access to `ctypes`, native extensions, or `os.fork()` could bypass the Python-level monkey-patching. True kernel-level isolation requires Linux network namespaces.

7. GAT Requires Labeled Training Data

The cybersecurity firewall and blame attribution features require a trained GNN. The current system uses synthetically generated graphs. Production deployment requires 30-90 days of real trace collection and root cause labeling.

8. Lexical False Positives in Semantic Circuit Breaker

The keyword `POST` in "POST-processing" or "HTTP POST is the correct method" would trip the circuit breaker. The AST scan mitigates this for Python code but not for natural language descriptions.

---

20. Conclusion

DriftGuard-X v2 represents a genuinely novel platform addressing the unsolved problem of causal root-cause attribution in AI agent pipelines.

Technical Achievements

Achievement	Metric
End-to-end test coverage	156+ tests, 25 E2E files
Novel subsystems implemented	14 (VTI, ARC, AOR, JIT, SCB, Merkle-DAG, Pre-emptive Shedding, GAT, BCRB, Policy, Ledger, Recovery, Rationale, Trace SDK)
Data model definitions	25+ Pydantic models, 11 ORM tables
Statistical algorithms	6 (EWMA, z-score, PSI, KS, JSD, CUSUM)
Lines of production code	~20,000+
Patent-ready claims	14 documented mechanisms

Honest Assessment

DriftGuard-X is **research-grade and enterprise-capable** in its core design patterns. The architecture is sound. The mathematical claims are bounded. The security posture is defensible.

The system is **not yet production-deployed** primarily because:

1. Recovery adapters are mocked (critical path gap).
2. OIDC and KMS integrations are stubs.
3. The GAT model requires training on real production data.
4. Infrastructure hardening (backup, HA, rate limiting) is incomplete.

With **15-20 weeks of focused engineering effort** across the 20 tasks above, DriftGuard-X is capable of reaching full production deployment.

The patent filing opportunity is substantial — 14 novel mechanisms are implemented and documented with running tests as evidence, and the `docs/` directory contains pre-structured evidence matrices ready for counsel review.

End of DriftGuard-X v2 Project Report
Document: `docs/PROJECT_REPORT.md`
Repository: `c:\Users\aniru\Downloads\driftguardx`
Version: 2.0.0-rc.1 | Report Date: 2026-07-25
Estimated length: ~13,000 words (~65 equivalent pages at 200 words/page)